Cómo mejoraría la captación de asistentes sin contratar más personal

Investigación para la prueba técnica de RENASER | Jarvis Miranda Mamani | 14 de octubre de 2026

Mi punto de partida es sencillo: antes de pensar en un evento con 3.000 personas, necesitamos saber si el equipo puede llevar a 30 asistentes reales con los recursos que ya tiene. Hoy hay 200 contactos, cuatro trabajadores, un CRM a medio implementar y un seguimiento que no está organizado. Ahí veo el primer problema que resolvería.

No empezaría haciendo una plataforma nueva. Primero ordenaría los contactos y su estado: interesado, inscrito, confirmado y asistió. Son cosas distintas. Tener una lista de 200 personas no significa que 200 quieran asistir, y enviar un mensaje tampoco significa que alguien haya confirmado.

Las tres alternativas que revisé

1. Seguir con el CRM e integrar n8n. Es la opción que escogería para el piloto. El formulario enviaría la información a un flujo que revise campos, evite duplicados, cree o actualice el contacto y deje una tarea de seguimiento. El CRM seguiría siendo el lugar donde el equipo consulta el estado oficial; n8n se encargaría del trabajo repetitivo. Tiene sentido porque no obliga a rehacer lo que ya existe. Antes de usarlo con personas reales tendría que comprobar la API del CRM, los permisos, los errores y los reintentos.

2. Mantener el CRM y utilizar Make. También permite conectar formularios y servicios sin programarlo todo. Puede ser más rápido si ya existe un conector listo para el CRM. El punto a vigilar es el consumo: Make incluye 1.000 créditos mensuales en su plan Free, y un recorrido con varios módulos puede consumir más de un crédito. No lo descartaría para siempre, pero probaría primero cuánto cuesta un registro completo y cómo recupera los fallos.

3. Google Sheets con Apps Script. Serviría para salir de una urgencia: lista de contactos, responsables y tareas pendientes. Sin embargo, cuando varias personas trabajan a la vez hay que cuidar la edición, los permisos y las duplicaciones. Además, Apps Script tiene cuotas de ejecución. Lo usaría como apoyo temporal, no como la base definitiva si queremos crecer.

Mi decisión

Me quedo con el CRM actual y n8n, siempre que la conexión sea viable. Es el camino que menos cambia la operación y deja a las cuatro personas trabajando sobre una misma lista. En esta primera semana utilizaría reglas claras para validar y asignar contactos; la IA quedaría para resumir mensajes o preparar borradores, siempre revisados. No hace falta poner un agente de IA donde basta una condición sencilla.

Qué recursos necesitaría

Separé los precios publicados de lo que yo presupuestaría para una prueba. No son una cotización de RENASER ni un gasto aprobado.

- n8n Cloud anuncia Starter desde 20 EUR/mes y Pro desde 50 EUR/mes, con facturación anual; los límites publicados son 2.500 y 10.000 ejecuciones respectivamente. Si cada registro activa un flujo, 3.000 registros ya pueden superar el primer límite, incluso antes de sumar recordatorios. [2]
- Alojar n8n Community por cuenta propia podría requerir, como estimación inicial mía, unos USD 10 a 25 mensuales para infraestructura básica. También hay que contar respaldos, mantenimiento y tiempo de administración.
- Make Free publica 1.000 créditos al mes. A modo de ejemplo, si cada contacto consumiera seis créditos, 200 contactos serían 1.200 créditos y 3.000 serían 18.000. Hay que medir el consumo real del escenario antes de tomar una decisión. [3]
- Para Sheets y Apps Script podríamos usar cuentas existentes, pero hay cuotas y gastos indirectos de soporte. El costo de correo, telefonía e IA depende del volumen y del proveedor; no lo consideraría gratis por defecto. [4]

Cómo organizaría los primeros siete días

No contrataría gente. Repartiría responsabilidades: una persona para integraciones y calidad, una para captación, otra para comunicaciones y seguimiento, y otra para operación y medición. Los nombres concretos se decidirían con la empresa.

Día 1. Revisar el CRM, permisos de contacto y los 200 registros. Detectar duplicados y saber qué datos faltan.

Día 2. Conectar un formulario de prueba, guardar el contacto en el sistema definido y comprobar que no se pierda ni se duplique.

Día 3. Crear tareas de seguimiento y comprobar un canal alternativo autorizado al de WhatsApp.

Día 4. Hacer una prueba controlada con contactos que aceptaron comunicaciones. Registrar respuestas y pendientes.

Día 5. Revisar errores, reintentos y la recuperación si el sistema se cae. Corregir lo que falle.

Día 6. Ensayar confirmaciones y control de asistencia. Preparar una lista operativa para el evento.

Día 7. Mostrar resultados a dirección y decidir si se amplía, se ajusta o se detiene el piloto.

Qué evitaría durante esa semana

No gastaría tiempo desarrollando otra web, montando varios agentes autónomos o llenando el calendario de publicaciones sin saber dónde se pierden los interesados. Tampoco enviaría mensajes masivos a contactos que no hayan autorizado ese canal. La prioridad es que cada persona tenga un estado, un responsable y un siguiente paso.

Para el piloto conectado al CRM, presupuestaría 24 a 36 horas técnicas y 12 a 16 horas del resto del equipo durante la semana. Son estimaciones propias, sujetas a accesos y calidad de datos; no son tiempos medidos en RENASER. La prueba ejecutada usó n8n Cloud sin activar pagos.

Qué mediría para saber si de verdad mejoró

Empezaría con cinco números que el equipo pueda revisar cada día: contactos válidos con permiso, nuevos inscritos, confirmaciones explícitas, asistentes comprobados y contactos que llevan demasiado tiempo sin seguimiento. También miraría el tiempo que dedica el equipo a tareas manuales y los errores de envío.

Para conseguir 30 asistentes desde 200 contactos haría falta una conversión final del 15 %. Es una referencia matemática, no algo que ya haya ocurrido. De la misma manera, si supusiéramos que el 25 % de los contactos se inscribe y que el 60 % de los inscritos asiste, llegar a 3.000 asistentes requeriría unos 20.000 contactos. Esos porcentajes todavía deben medirse en el piloto.

Qué se hizo para esta prueba

Construí y ejecuté el piloto en n8n Cloud con datos ficticios y almacenamiento en Data Tables. Valida el registro, busca duplicados por evento y correo, clasifica por reglas, registra un seguimiento simulado y actualiza la misma fila. No se enviaron correos ni mensajes reales.

Comprobé cuatro ejecuciones: Andrea se registró en la fila 1; al repetirla conservó la fila y un intento. Bruno quedó en pendiente_revision, responsable D; al recuperarlo conservó la fila 2 y llegó a dos intentos. La tabla mantuvo dos filas al recargarla. Falta integrar formulario, CRM y canales reales, y probar bajas y concurrencia. Las 17 pruebas de Node.js son un respaldo separado; no certifican n8n.

Riesgos que no dejaría sin atender

Si WhatsApp deja de funcionar: mantendría las tareas registradas y utilizaría correo o llamadas solo con autorización. Cuando vuelva el servicio revisaría las tareas antes de reintentar, para no enviar mensajes repetidos.

Si hay duplicados o datos incorrectos: detendría la ampliación, revisaría qué parte del recorrido los produce y haría una conciliación con el CRM.

Si los contactos no confirman: no aumentaría la publicidad a ciegas. Revisaría los mensajes, tiempos de respuesta y motivos de abandono con los cuatro responsables.

Conclusión

Mi recomendación es comenzar pequeño y medir bien. Si el equipo consigue organizar los 200 contactos, seguirlos sin pérdidas y comprobar cuántos llegan al evento, tendremos datos para decidir qué automatizar después. Recién con ese resultado propondría el presupuesto y la capacidad necesaria para acercarnos a los 3.000 asistentes.

Fuentes revisadas

n8n, documentación del nodo Webhook: https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/
n8n, precios y límites: https://n8n.io/pricing/
Make, precios y créditos: https://www.make.com/pricing
Google, cuotas de Apps Script: https://developers.google.com/apps-script/guides/services/quotas
Supabase, precios y límites (alternativa no elegida): https://supabase.com/pricing

n8n, Data Tables (almacenamiento del piloto): https://docs.n8n.io/build/work-with-data/data-tables/